

Docker Workshop

Lab 2

Working with containers

1	COMMANDS COVERED	1
2	RUNNING CONTAINERS FROM IMAGES	3
2.1	RUNNING A CONTAINER WITHOUT AN EXISTING IMAGE	3
2.2	RUNNING A CONTAINER WITH AN INTERACTIVE SHELL	4
2.3	RUNNING A NAMED CONTAINER IN DETACHED MODE.....	5
2.4	STARTING AND STOPPING CONTAINERS	6
2.5	ATTACHING TO RUNNING CONTAINERS	7
2.6	EXECUTING A NEW PROCESS WITHIN AN EXISTING CONTAINER	7
3	REMOVING CONTAINERS.....	8
3.1	AUTO REMOVAL WITH THE --RM FLAG	8
3.2	REMOVING STOPPED / RUNNING CONTAINERS	9
4	INTERACTING WITH THE CONTAINER FILE SYSTEM	10
5	RETRIEVING CONTAINER LOGS	11
6	INSPECTING CONTAINERS.....	12

1 Commands covered

Running, starting and stopping containers	Listing containers
<pre>docker container run image-name</pre> <pre>docker container run -it image-name application-name</pre> <pre>docker container run -d --name container-name image-name</pre> https://linuxize.com/post/docker-run-command/ https://phoenixnap.com/kb/docker-run-command-with-examples <pre>docker container stop container-name/id</pre> <pre>docker container start container-name/id</pre>	<pre>docker container ls</pre> <pre>docker container ls -a</pre> <pre>docker container ls --filter status=exited running</pre> <pre>docker container ls --filter name=xxx</pre> <pre>docker container ls --filter id=xxx</pre> https://linuxize.com/post/how-to-list-docker-containers/ https://www.baeldung.com/ops/docker-list-containers

<pre>docker container restart container-name/id</pre> https://www.thegeekdiary.com/how-to-list-start-stop-delete-docker-containers/ https://eldermoraes.com/docker-basics-how-to-start-and-stop-containers/	

Attaching to a running container	Interacting with the container file system
<pre>docker container attach container- name/id docker container exec -it container-name/id shell-name</pre> https://linuxize.com/post/how-to-connect-to-docker-container/ https://www.baeldung.com/ops/docker-attach-detach-container	<pre>docker container cp local-file container-name:container- directory docker container cp container- name:container-file local- directory</pre> https://www.cloudsavvyit.com/13987/how-to-use-docker-cp-to-copy-files-between-host-and-containers/ https://www.baeldung.com/ops/docker-copying-files

Retrieving container logs	Inspecting the container
<pre>docker container logs -f -t container-name/id docker container logs --tail x container-name/id</pre> https://geekflare.com/check-docker-logs/ https://sematext.com/guides/docker-logs/	<pre>docker container stats container- name/id docker container diff container- name/id docker container inspect container-name/id docker container inspect --format Go-template container-name/id</pre> https://www.cloudsavvyit.com/13715/how-to-monitor-the-resource-usage-of-docker-containers/ https://semaphoreci.com/blog/container-diff-tutorial https://www.thegeekdiary.com/how-to-get-information-about-a-container-in-docker/

	https://www.ctl.io/developers/blog/post/what-to-inspect-when-youre-inspecting

Creating an image from a container	Restart policies for containers
<pre>docker container commit -m message -a author container- name/id new-image-name</pre> https://adamtheautomator.com/docker-commit/ https://phoenixnap.com/kb/how-to-commit-changes-to-docker-image	<pre>docker container run --restart always unless-stopped on- failure image-name</pre> https://www.cloudsavvyit.com/10912/how-to-use-docker-restart-policies-to-keep-containers-running/

Removing containers	
<pre>docker container rm container- name/id docker container rm -f container- name/id docker container rm -f \$(docker container ls -a -q) docker container prune</pre> https://linuxize.com/post/how-to-remove-docker-images-containers-volumes-and-networks/ https://phoenixnap.com/kb/remove-docker-images-containers-networks-volumes	

2 Running containers from images

2.1 Running a container without an existing image

Type:

```
docker run hello-world
```

This attempts to start up a container from an image called `hello-world` which [is hosted here](#). Notice that Docker informs us that it is not able to find the image in the local registry, and it then

makes an attempt to locate and pull this image down from the default registry (Docker Hub). Since we did not specify a tag for this image, Docker automatically appends the tag `latest`.

The container for this very basic image simply displays some output message from its execution and exits immediately to the command line. This image is typically used to create a container immediately after a new Docker installation to ensure that it is running properly on the native host machine.

You can verify that the image has been loaded to the local registry with:

```
docker images
```

To see the list of running containers, type:

```
docker ps
```

Nothing is listed because the container has already completed executing. To see the list of all containers, including those that have stopped, type:

```
docker ps -a
```

IMPORTANT NOTE: All command options are either one letter preceded by a `-`, or a complete word preceded by two dashes `--`. You can check this out with `docker ps --help`. Thus, the option `-a` is exactly same as `--all`.

However there are some options for all commands which do not have an equivalent one letter shortcut. For example, for the case of `docker ps`, we must specify the options `--format` and `--no-trunc`.

We can see the container ID (which is conceptually similar to the image ID and is used to uniquely identify each running container), the image used to create it, the command it last ran when it was created, its exit status (0 - for a normal exit), ports and container names.

If we run a container without specifying a name, Docker automatically generates a random name for us, so you will often see a lot of these random names in container listing output.

You should also be able to view listing of containers (active as well as stopped) via the UI of your specific Docker installation

2.2 Running a container with an interactive shell

Let's pull the latest `alpine` image from Docker Hub.

```
docker pull alpine
```

Next, let's run a container from this downloaded image and open an interactive terminal shell in the running container:

```
docker run -it alpine /bin/sh
```

The `-i` flag keeps STDIN open from the container, even if we're not attached to it. The `-t` flag tells Docker to assign a pseudo-tty to the container we're about to create. This provides us with an interactive shell in the new container. The `/bin/sh` is a reference to the system shell. For the `alpine` container, this will be the `ash` shell that is typically used in embedded Linux systems: it is a lighter weight version of the standard `bash` shell found in Ubuntu and Debian.

Check the contents of the current directory with the standard Linux command

```
/ # ls -la
total 64
drwxr-xr-x  1 root    root      4096 Jul 14 08:25 .
drwxr-xr-x  1 root    root      4096 Jul 14 08:25 ..
-rwxr-xr-x  1 root    root         0 Jul 14 08:25 .dockerenv
drwxr-xr-x  2 root    root      4096 Jun 15 14:34 bin
drwxr-xr-x  5 root    root       360 Jul 14 08:25 dev
drwxr-xr-x  1 root    root      4096 Jul 14 08:25 etc
drwxr-xr-x  2 root    root      4096 Jun 15 14:34 home
.....
```

AS you can see, it's the typical layout of the root directory of a Linux file volume. Check the account you are logged in under as well as the hostname of this container.

```
/home # whoami
root

/home # hostname
99e8416f67c6
```

If you start up another Powershell terminal, and type

```
docker ps
```

you should be able to see the running container listed with the hostname being the same as the container ID.

You should also be able to see this container listed as being active in the UI of your specific Docker installation

In the interactive shell of the running container, exit by typing:

```
/ # exit
```

When you started that container, the system shell (`/bin/sh`) was the one and only process running inside the container. The moment you exit or kill this main process, the container will stop running as well.

Check for this with:

```
docker ps -a
```

2.3 Running a named container in detached mode

In the other PowerShell window, start up another container from the same image but with an explicitly assigned name (using the `--name` flag) and run it in detached mode (using the `-d` flag).

```
docker run -d --name superman -it alpine
```

This command exits successfully with the container ID of the newly created container.

The `-it` flag ensures that there is an interactive shell started within the container even though we will be detached from it.

The `-d` flag detaches the client while the container's foreground process continues.

It is common to start containers in detached mode so that we can return back to the Powershell to continue typing more commands. Also, as we will see later, many containers will have an app within them (such as a web app) that starts running when the container starts and there is no need for us to perform any further action after running the container.

We can actively connect (or attach) to this running container at a later time via a interactive shell terminal if we wish, and we will see how to do that later.

Keep in mind that container names that you assign have to be unique. If we try to create two containers with the same name, the command will fail. We need to delete the previous container with the same name before we can create a new one.

If you check the list of running containers again, you should see this container with our specified name actively running

```
docker ps
```

You can start as many containers as you want from a single image in this manner. All these containers will be based from the same image with their own individual read-write layers on the top to hold state changes related to changes in the file system of their respective containers.

2.4 Starting and stopping containers

You can start the previously stopped / exited container with:

```
docker start container-name/id
```

When you start a stopped container, it automatically starts in detached mode, so you will not have an interactive terminal inside it although it remains running in the background.

If you use the *container-id* instead of *container-name*, you only need to specify enough letters from the ID to uniquely identify that container from other available containers in the listing. Usually 3 to 4 letters is adequate in nearly all cases, and often times 1 – 2 letters will suffice.

You can stop any running container with:

```
docker stop container-name/id
```

You can also restart a running container (stop followed immediately by start) with:

```
docker restart container-name/id
```

NOTE: Certain basic images (such as `hello-world`) when started as a container will run and then immediately exit. They will not stay active and running such as the `alpine` containers.

You can also start and stop containers using the UI of your specific Docker installation

2.5 Attaching to running containers

You had previously started a container named `superman`. If it has stopped, start it again

You can now attach to the shell within the container with:

```
docker attach superman
```

Just like `start` / `restart` command previously, you can either use the container name or id for the command above.

You can then type standard Linux commands into the interactive shell as we have seen previously. If you now type `exit` in the interactive shell, the container will exit as explained.

Start it again and attach to it:

```
docker start superman
```

```
docker attach superman
```

Now this time in the interactive shell, type `Ctrl-PQ` which allows you to exit without terminating the main process and so the container still remains running (check with `docker ps`)

2.6 Executing a new process within an existing container

We can execute a new process / application within an existing container:

```
docker exec -it container-name/id /bin/sh
```

In the above command, we execute the `/bin/sh` process in the specified container.

If you do this, and then you check for running processes within the shell, you will notice that there are two shells now running:

```
/ # ps
PID   USER     TIME   COMMAND
  1  root      0:00   /bin/sh
 30  root      0:00   /bin/sh
 39  root      0:00   ps
```

This is because the `exec` command creates a new shell process, rather than attaching to the existing shell process in the container.

This means that if you exit the shell process now (by typing `exit` at the prompt), the container still remains running because there is still a single shell process still left running in it.

Again, check this with `docker ps`

You can attach to a shell within a running container using the VS Code Docker extension UI.

3 Removing containers

Stopping containers does not remove them. If you wish to remove a container permanently from the host machine, you will first have to stop them.

Identify a running container (start one if there is none running at the moment), and attempt to remove it with:

```
docker rm container-name/id
```

Notice that you will get an error message.

Identify a stopped container (stop one if all are running) and then attempt to remove it with the same command: this time you should succeed.

You can add the `-f` flag to force the remove of a running container.

Identify another running container (start one if there is none running at the moment), and attempt to remove it with:

```
docker rm -f container-name/id
```

This time you should succeed.

3.1 Auto removal with the `--rm` flag

So far, we have seen that containers that we have started and subsequently stopped still exist and can be seen in the listing of containers.

For e.g.

```
docker run -d --name hulk -it alpine
```

```
docker stop hulk
```

```
docker ps -a
```

This means that if we subsequently no longer have any use for this container, we would have to explicitly delete it with a further command:

```
docker rm hulk
```

A common Docker operation is to only run the container once to perform a specific operation within it (for e.g. running a script or process) and we do not subsequently expect to start it and run it again. We will be seeing this in subsequent labs.

For this we can add the `--rm` flag to automatically remove the container when it stops, which is very useful so that you don't manually have to clean up all these stopped containers periodically in order to not clutter your listing and free up system resources.


```
docker run --rm alpine /bin/sh -c "echo Demo simple command; ls -la"
docker ps -a
```

The VS Code Docker extension UI allows you to run containers from images in either detached mode or interactive mode, but in both cases the containers are started with the `--rm` option which means they will be removed as soon as they stop.

However, for existing containers that were started independently from the VS Code Docker Extension UI, there is also functionality to stop, start and restart containers as well.

3.2 Removing stopped / running containers

Start up some containers and leave a few others stopped.

To remove all stopped containers, you can type:

```
docker container prune
```

Respond `y` to the prompt that is presented.

Stop some of the running containers and leave a few others running to test out the next command.

To remove all containers (both running and stopped), type:

```
docker rm -f $(docker ps -a -q)
```

The first part of this command (`docker ps -a -q`) obtains the container IDs of all the containers and then passes it to the second command (`docker rm`) which has a `-f` flag to force removal regardless of whether the container is running or stopped.

As this is a forceful removal of every container visible to the Docker Daemon, it may accidentally remove containers with relevant applications, databases and other work belonging to other users on a shared host machine. Therefore, use this **WITH CAUTION**

Check again that all containers have been removed

```
docker ps -a
```

You can also delete containers via the UI of your specific Docker installation (Docker Desktop / Rancher Desktop)

You can either select and delete a single container or all stopped containers using the VS Code Docker extension UI.

Experiment starting, stopping and deleting with either of these UIs.

4 Interacting with the container file system

If you had previously removed the container called `superman`, create a new container with the same name with an interactive shell open in it:

```
docker run --name superman -it alpine /bin/sh
```

Create a new directory and store a simple file in it with these commands and then exit:

```
/ # cd home
/home # mkdir mystuff
/home # cd mystuff
/home/mystuff # echo "Hello world" > message.txt
/home/mystuff # cat message.txt
Hello world
/home/mystuff # exit
```

Start the container again and attach to it in a shell terminal.

```
docker start superman
```

```
docker attach superman
```

Verify that the file and directory you created there has still persisted:

```
/ # cd home/mystuff
/home/mystuff # cat message.txt
Hello world
/home/mystuff # exit
```

If you have the VS Code Docker extension installed, you can utilize its UI to navigate through and view the file system of a running container and the contents of the files within it.

This demonstrates that any state changes made to a container (i.e. changes to its file system) between successive stops and starts are preserved. However, once a container is removed and then restarted from an existing image, all state changes are permanently lost. The only way to preserve them is through volumes and bind mounts (which we will be examining in an upcoming lesson).

In any empty directory on your host machine file system, create a file: `things.txt` and populate it with random content. Switch to this directory in a PowerShell terminal, and type this command to copy this file into the `/home/mystuff` directory in the `superman` container.

```
docker cp things.txt superman:/home/mystuff
```

Notice that the command above works regardless of whether the container is running or has stopped.

Start the container again and attach to it in a shell terminal.

```
docker start
```

```
docker attach superman
```

Verify that you have transferred this file from the host file system to the container file system successfully:

```
/ # cd home/mystuff
/home/mystuff # ls -l
total 8
-rw-r--r--    1 root    root          12 Jul 14 09:31 message.txt
-rwxr-xr-x    1 root    root          17 Jul 14 11:18 things.txt
/home/mystuff # cat things.txt
my things to do
```

If you have the VS Code Docker extension installed, you can utilize its UI to navigate through and view the file system of a running container and the contents of the files within it.

In a Powershell terminal, transfer the `message.txt` file that you created earlier within the container to any directory on your host machine file system (replace *local-directory* appropriately, for e.g. `C:\data`).

NOTE: If your Windows file path contains spaces, ensure that you enclose it within double quotes so that Docker interprets it correctly as single argument, for e.g.:

```
"C:\Users\James Bond\Desktop\cool stuff"
```

```
docker cp superman:/home/mystuff/message.txt local-directory
```

A shortcut to transfer to the directory that the Powershell terminal is in is to use the `.`

```
docker cp superman:/home/mystuff/message.txt .
```

Verify that the file has been transferred successfully using Windows File Explorer.

You can now stop the container through the UI or using the `exit` command in the interactive shell terminal.

Lastly, start another container with any random name from the same image that you started the `superman` container from:

```
docker run --name batman -it alpine /bin/sh
```

In the shell, navigate to the home directory and check its contents:

```
/ # cd home
/home # ls -l
total 0
/home # exit
```

This clearly demonstrates that although these 2 containers are started from the same image (`alpine`), their state of their file system is unique to each of them and is maintained in the top level read-write layer that builds on top of the lower level read-only base image layers that are common to both containers.

5 Retrieving container logs

Here, we simulate the situation of continuous console output from a process inside a container by running a container with a script that runs in an endless loop. In a real-life app (which we will see later), there may be continuous console output from a process such as web server application running within a container and we may wish to inspect this console output.

We start another container based on the Ubuntu image with a script that runs endlessly and outputs a console output statement every 1 second:

```
docker run --name spiderman -d ubuntu /bin/bash -c 'count=1; while true; do echo Counting $count; ((count++)); sleep 1; done'
```

To get a log of all the console output to the current point of time:

```
docker logs spiderman
```

Issue the command above after a random period of time to see how the console log output has changed.

We can also follow the log output dynamically (-f) with timestamps added in (-t)

```
docker logs -f -t spiderman
```

Press Ctrl-C to exit

To see the last 5 console output statements with timestamps added:

```
docker logs --tail 5 -t spiderman
```

If you have the VS Code Docker extension installed, you can utilize its UI to view the logs of a running container.

6 Inspecting containers

To list all the active processes inside a running container

```
docker top spiderman
```

Here we should be able to see the single Bash shell process running the script that we started the container with earlier. In addition, if you had utilized the VS Code Docker extension to view logs or connect to the container, you may also see other processes as well in the container.

To get a live stream of container resource usage statistics: CPU, memory usage, network I/O performance, we can run:

```
docker stats spiderman
```

Press Ctrl-C to exit.

To inspect changes to a container's file system from the time it was started, we can use the superman container as an example where earlier we had made a number of changes to the file system:

```
docker diff superman
```

The output at the top should look similar to the following:

```
C /home
A /home/mystuff
A /home/mystuff/message.txt
A /home/mystuff/things.txt
C /root
A /root/.ash_history
```

A, C, R in the output above stands for added / changed / removed

If you have used VS Code Docker extension earlier, you will additionally see a lot of other changes related to processes that were run in the container to facilitate the extension use.

To obtain information on configuration details, commands, networking configuration, etc, we can type:

```
docker inspect superman
```

This will be useful in later labs when we need to get more detailed info on a running container.

To drill down into the detailed list of information output, we can use a Go template together with the `--format` option. For example:

```
docker inspect --format '{{ .State.Running }}' superman
```

```
docker inspect --format '{{ .NetworkSettings.IPAddress }}' spiderman
```

If you have the VS Code Docker extension installed, you can utilize its UI to inspect the container to get this information as well.

You can stop any running containers using the UI or through the `docker stop` command.